

Queues

LEARNING OBJECTIVES

After studying this chapter, readers will be able to:

- Understand queue and its operations
- Implementation of queue using list
- Implementation of queue using dictionary
- Implementation of queue using class
- Implementation of queue using queue module
- Understand circular queue
- Implementation of circular queue using class
- Understand double-ended queue
- Implementation of double-ended queue
- Understand a priority queue
- Implementation of priority queue using class
- Implementation of priority queue by importing PriorityQueue package of queue module
- Implementation of priority queue using heapq module

Linear data structures are divided into arrays, stacks and queues. In this chapter we will focus on the queue as a linear data structure. Queue data structures can be implemented using list, dictionary, class and the queue module. Simple examples of a queue are waiting for a bus in a bus stand and jobs that are taken for printing by a printer.

6.1 QUEUE

A queue is simply a waiting line found at railway stations, airports, cinema halls, etc. It grows when people join the queue and shrinks when people leave the queue. Unlike a stack, a queue is a data structure in which both ends are used: one for adding new elements and the other for removing them. The end where the elements join the queue is called the *rear* and the

end from where the elements are removed is called the *front*. A queue follows the FIFO (first in, first out) discipline (Figure 6.1).

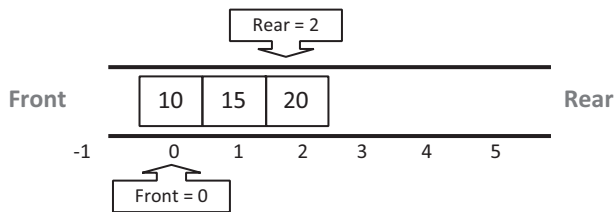


Figure 6.1 Sample queue

6.2 QUEUE OPERATIONS

The operations in Table 6.1 are used to manage a queue.

Table 6.1 Queue operations

Operation	Meaning
isEmpty()	Checks if the queue is empty
isFull()	Checks if the queue is full
enqueue(el)	Put the element at the end of the queue
dequeue()	Removes the first element in the queue
first()	Returns the first element in the queue without removing it

A series of enqueue and dequeue operations are shown in Figures 6.2–6.7e.

6.2.1 Enqueue (enter data into a queue)

The *enqueue(Q)* operation inserts an element at the rear of the queue *Q*. It follows the following steps:

1. Let *rear* be the index of the last element.
2. Check if the queue is full (if it is a restricted queue).
3. If the queue is not full:
 - a. Increment *rear* by one.
 - b. Assign the data at the *rear* index of queue.
 - c. Return queue.
4. Else report failure (*queue overflow*).

The maximum size of a Python list on a 32 bit system is around 536,870,912 elements. Since we are using a list to implement a queue, the condition *isFull()* is not required. If we implement a restricted list (a list with a

predefined size), then *isFull()* is required to be implemented. We increment the ‘top’ by one and insert $Q[\text{rear}]$ into the queue (Figures 6.2 to 6.6).

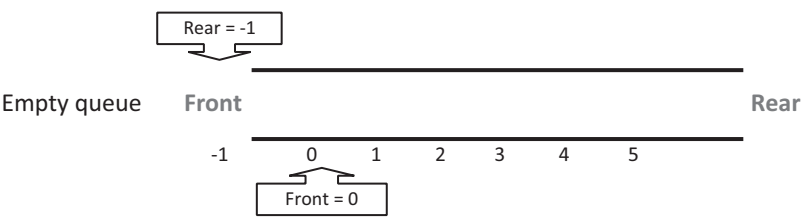


Figure 6.2 Empty queue

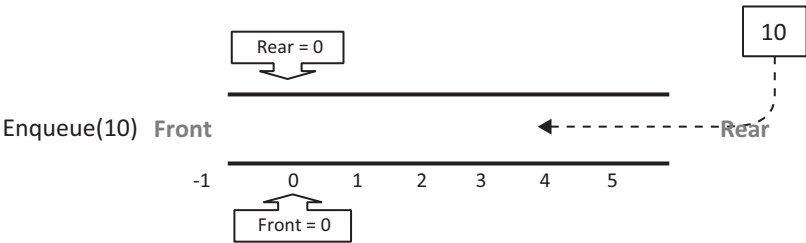


Figure 6.3 Before inserting 10 into the queue

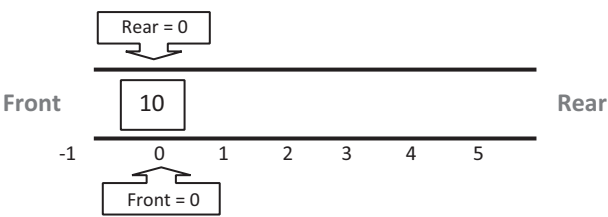


Figure 6.4 After inserting 10 into the queue

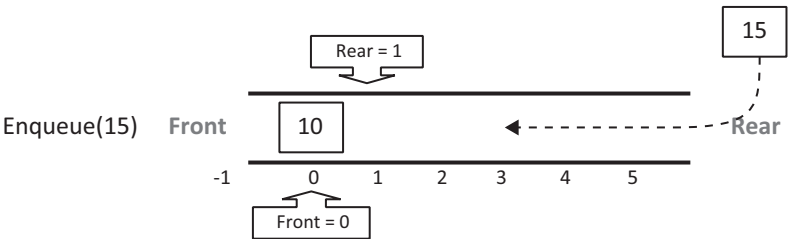


Figure 6.5 Before inserting 15 into the queue

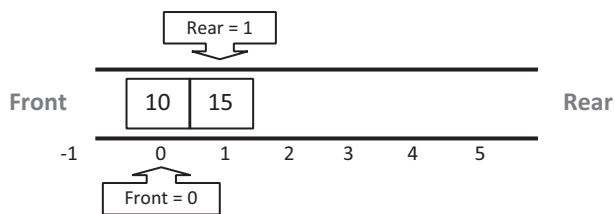


Figure 6.6 After inserting 15 into the queue

6.2.2 Dequeue (delete data from a queue)

We execute the following steps to perform the delete operation (Figures 6.7a to 6.7e):

1. Let x be the variable to store the deleted element.
2. If the queue is not empty:
 - a. Set x = front index of the queue.
 - b. Increment the front by one.
 - c. Return queue.
3. Else report failure (*queue underflow*).

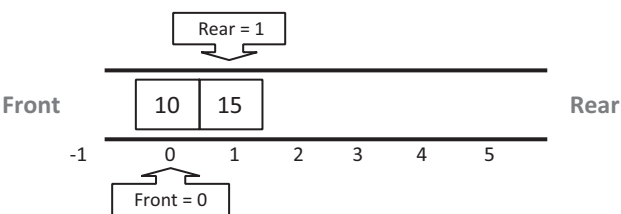


Figure 6.7a Initial queue

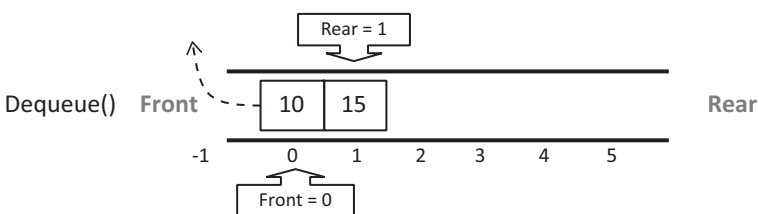


Figure 6.7b Before deleting 10 from the queue

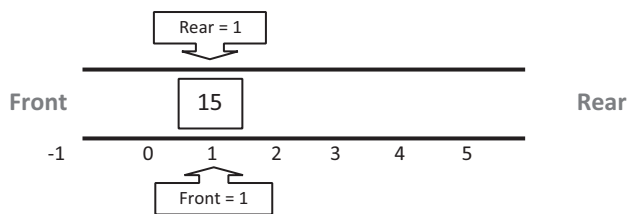


Figure 6.7c After deleting 10 from the queue

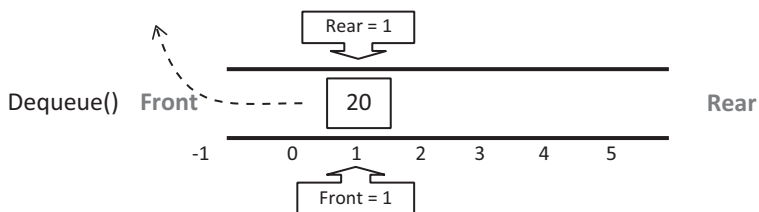


Figure 6.7d Before deleting 15 from the queue

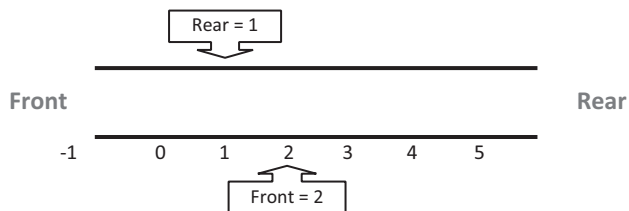


Figure 6.7e After deleting 15 from the queue

Since elements are entered at one end and removed from the other end, both ends are to be monitored.

Consider the ordered list $L = [a_1, a_2, a_3, \dots, a_n]$. If we assume L represents a queue, a_1 is in the front end, a_n is at the rear and a_i is behind a_{i-1} .

Queues are very important data structures in time-sharing and distributed network systems where the users try to access the system resources simultaneously.

6.3 QUEUE (ADT)

The following are the operations and the set of axioms on the data structure queue.

Operations: See Table 6.2.

Table 6.2 Queue ADT operations

<i>Operation</i>	<i>Meaning</i>
Create()	Creates an empty queue
Add(<i>el</i> , <i>Q</i>)	Adds the element “ <i>el</i> ” to the queue
Delete(<i>Q</i>)	Deletes the element from the front
getFront()	Outputs the front element without removing it
isEmpty()	Returns “true” if the queue is empty; “false” otherwise
isFull()	Returns “true” if the queue is full; “false” otherwise

Axioms: See Table 6.3.

Table 6.3 Queue ADT axioms

<i>Axiom</i>	<i>Returned value</i>
isEmpty(create())	True
isEmpty(Add(<i>el</i> , <i>Q</i>))	False
deleteQ(create())	Error
getFront(create())	Error
getFront(Add(<i>el</i> , <i>Q</i>))	<i>el</i> if isEmpty(<i>Q</i>); else getFront(<i>Q</i>)

Axiom 1 says that when we create a queue, it is initially empty. However, after adding an element to a queue, obviously the queue is not empty, which is indicated by axiom 2. We cannot delete an element from an empty queue and thus deleteQ(create()) returns an error and so is the case with getFront(create()). Axiom 5 is obvious since when we add an element it joins the queue in the rear and thus getFront(Add(*el*,*Q*)) is the same as getFront(*Q*) unless *el* is added to an empty queue, in which case rear = front.

6.4 IMPLEMENTATION

There are various ways to implement a queue. It can be implemented using a list, dictionary or a class in Python.

6.4.1 Implementation using a list

The elements of the queue are the elements of a list. As a list is not a fixed-size data structure, while implementing, the queue is also not fixed size. But

we can restrict the size of the list, hence the size of the queue can be made fixed.

Now let us implement a queue using a list without a restriction of upper limit or the size of the queue.

Python program 1

```
# To create an empty queue
def create():
    Q=[]
    front = 0
    rear = -1
    print("Empty Queue created")
    return Q,rear,front
```

The create() method creates an empty queue with the front pointing to 0 and rear pointing to -1. Here Q = [] indicates an empty list.

Program continues...

```
# To check if the queue is empty or not
def isEmpty(front,rear):
    if (front > rear or rear == -1):
        front = 0
        rear = -1
        return 1
    else:
        return 0
```

The isEmpty() method checks if the queue is empty. It checks if the front is greater than the rear or the rear is at -1. It returns 1 to indicate the queue is empty; otherwise it returns 0 to indicate the queue is not empty (at least one element is there in the queue).

Program continues...

```
# To insert an element into a queue
def enqueue(Q,rear):
    no = int(input(("Enter a number to push onto
        the Queue ")))
    rear += 1
    Q.append(no)
    return Q,rear
```

The enqueue() method takes two parameters: the queue and the rear. Every time an element is inserted into the queue, the rear is incremented by one and the element is appended to the list. As a list is a heterogeneous data structure, we force the user to enter an integer only.

Program continues...

```
# To delete an element from a queue
def dequeue(Q, front, rear):
    if (isEmpty(front, rear) == 1):
        front = 0
        rear = -1
        print("Underflow: There is no element in the Queue")
        return Q, front, rear
    else:
        print("The element to pop is ", Q[0])
        return Q, front+1, rear
```

The dequeue() method takes three parameters: the queue, the front and the rear. Every time an element is deleted from the queue, it checks if the queue is empty. If the queue is empty, it shows an appropriate message, and if the queue is not empty, then the front of the queue is printed and the front is decremented by one. At last, the front and the rear of the queue is returned.

Program continues...

```
#To display elements of a queue
def display(S, front, rear):
    print("Front is at index -> ", front)
    print("Rear is at index -> ", rear)
    if (isEmpty(front, rear) == 1) :
        print("\nQueue is Empty")
    else:
        print("The elements in the Queue is")
        for i in range(front, rear+1):
            print(i, "->", Q[i])
```

The display() method takes three parameters: the queue, the front and the rear. This function prints the elements in the queue (the element inserted first is printed first).

Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert to Queue ")
    print("2. Delete from Queue ")
    print("3. Display the Queue")
    print("4. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

The menu() method shows a menu to perform the operations defined for a queue. It returns a choice of operation to perform.

Program continues...

```
if __name__ == "__main__":
    Q,rear,front=create()
    i=1
    while (i > 0 and i <=3 ):
        i = menu()
        if i == 1:
            Q,rear = enqueue(Q,rear)
            display(Q,front,rear)
        elif i == 2:
            Q,front,rear = deque(Q,front,rear)
            if (front > rear):
                front = 0
                rear = -1
            if (type(front) != str):
                display(Q,front,rear)
        elif i == 3:
            display(Q,front,rear)
        else:
            print("Exit")
```

Output

Empty Queue created

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to push onto the Queue 10

Front is at index -> 0

Rear is at index -> 0

The elements in the Queue is

0 -> 10

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to push onto the Queue 20

Front is at index -> 0

```
Rear is at index -> 1
The elements in the Queue is
0 -> 10
1 -> 20

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to push onto the Queue 30
Front is at index -> 0
Rear is at index -> 2
The elements in the Queue is
0 -> 10
1 -> 20
2 -> 30

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to pop is 10
Front is at index -> 1
Rear is at index -> 2
The elements in the Queue is
1 -> 20
2 -> 30

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to pop is 10
Front is at index -> 2
Rear is at index -> 2
The elements in the Queue is
2 -> 30

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to pop is 10
```

```

Front is at index -> 0
Rear is at index -> -1

Queue is Empty

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit

```

Now let us implement a queue using a list with a restriction of an upper limit or the size of the queue. There is no change in the `create()`, `dequeue()`, `isEmpty()` and `menu()` functions and some changes in the `enqueue()` and `display()` functions. A new function `isFull()` is also introduced. We are discussing only those functions that have some changes to Python program 1.

Python program 2

```

# To check the Queue is full
def isFull(rear,front,size):
    if (rear >= size-1):
        return 1
    else:
        return 0

```

The `isFull()` method checks if the queue is full. It checks if the $(\text{rear} - \text{front} + 1)$ is greater than the $(\text{size} - 1)$. It returns 1 to indicate the queue is full; otherwise returns 0 to indicate the queue is not full.

Program continues...

```

# To insert an element into a queue
def enqueue(Q,rear,front,size):
    if (isFull(rear,front,size) == 1):
        print("Overflow: No more place for the Queue")
        return Q,rear
    no = int(input(("Enter a number to insert into
the Queue ")))
    rear += 1
    Q.append(no)
    return Q,rear

```

The `enqueue()` method takes four parameters: the queue, the front, the rear and the size. Every time an element is inserted into the queue, it checks if the queue is full. If the queue is full, it shows an appropriate message; otherwise the rear is incremented by one and the element is appended to the list.

Program continues...

```
# To display the elements of a Queue
def display(Q,front,rear):
    print("Front is at index -> ",front)
    print("Rear is at index -> ",rear)
    if (isEmpty(front,rear) == 1) :
        print("\nQueue is Empty")

    elif(isFull(rear,front,size)==1):
        for i in range(front,rear+1):
            print(i, "->",Q[i])
        print("\nQueue is Full")

    else:
        print("The elements in the Queue is")
        for i in range(front,rear+1):
            print(i, "->",Q[i])
```

The display() method takes three parameters: the queue, the front and the rear. This function prints the elements in the queue (the element inserted first is printed first) if the queue is neither full nor empty.

Output

```
Enter the size of the Queue 3
Empty Queue created

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
Front is at index -> 0
Rear is at index -> 0
The elements in the Queue is
0 -> 10

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 20
Front is at index -> 0
Rear is at index -> 1
```

The elements in the Queue is

0 -> 10

1 -> 20

~~~ MENU ~~~

1. Insert to Queue

2. Delete from Queue

3. Display the Queue

4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 30

Front is at index -> 0

Rear is at index -> 2

0 -> 10

1 -> 20

2 -> 30

Queue is Full

~~~ MENU ~~~

1. Insert to Queue

2. Delete from Queue

3. Display the Queue

4. Exit

Enter a valid menu item ... 2

The element to delete is 10

Front is at index -> 1

Rear is at index -> 2

1 -> 20

2 -> 30

Queue is Full

~~~ MENU ~~~

1. Insert to Queue

2. Delete from Queue

3. Display the Queue

4. Exit

Enter a valid menu item ... 1

Overflow: No more place for the Queue

Front is at index -> 1

Rear is at index -> 2

1 -> 20

2 -> 30

Queue is Full

~~~ MENU ~~~

1. Insert to Queue

2. Delete from Queue

3. Display the Queue

```
4. Exit
Enter a valid menu item ... 2
The element to delete is 10
Front is at index -> 2
Rear is at index -> 2
2 -> 30

Queue is Full

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to delete is 10
Front is at index -> 0
Rear is at index -> -1

Queue is Empty

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit
```

6.4.2 Implementation of queue using a dictionary

Now let us implement a queue using a dictionary with a restriction of upper limit or the size of the queue. We already know that a dictionary is a key-value pair. We are implementing the queue with size restriction.

Python program 3

```
# To create an empty queue
def create():
    Q={}
    front = 0
    rear = -1
    print("Empty Queue created")
    return Q,front,rear
```

In the create() method, we have created an empty dictionary Q={} and assigned the front to 0 and rear to -1.

Program continues...

```
# To check the queue is Full
def isFull(front,rear,size):
    if (rear >= size-1):
        return 1
    else:
        return 0
```

Just like the previous program, the *isFull()* function checks if the queue is full. When the rear is greater than or equal to (size -1), it returns 1 indicating the queue is full; otherwise it returns 0 indicating the queue is not full.

Program continues...

```
# To check the queue is Empty
def isEmpty(front,rear):
    if (front > rear or rear == -1):
        return 1
    else:
        return 0
```

The *isEmpty()* function checks if the queue is empty. It returns 1, if the front is greater than rear or rear = -1. It is exactly the same as the previous Python program.

Program continues...

```
# To insert into the queue
def enqueue(Q,front,rear,size):
    if (isFull(front,rear,size) == 1 ):
        print("Overflow: No more place in the Queue")
    else:
        no = int(input(("Enter a number to insert into the Queue ")))
        rear += 1
        Q[rear] = no
    return Q,rear
```

The *enqueue()* function inserts an element into a queue. It takes four parameters – the queue, the front, the rear and the size – and returns the queue and rear. The working is the same as the Python program 2 described earlier.

Program continues...

```
# To delete from the queue
def dequeue(Q, front, rear):
    if (isEmpty(front, rear) == 1):
        print("Underflow: There is no element in the Queue")

    else:
        print("The element to delete is ", Q[front])
        del Q[front]
        front += 1
    return Q, front
```

The dequeue() function deletes an element. If there are no elements in the queue, it returns an appropriate message. Otherwise, it deletes the element that is at the front and increments the front by one and returns the queue and the front.

Program continues...

```
# To display the elements of the queue
def display(Q, rear, size):
    print("Front is at index -> ", front)
    print("Rear is at index -> ", rear)
    if (isEmpty(front, rear) == 1) :
        print("\nQueue is Empty")

    elif (isFull(front, rear, size) == 1):
        print("The elements in the Queue is")
        for i in list(Q.keys()):
            print(i, "->", Q[i])
        print("\nQueue is Full")

    else:
        print("The elements in the Queue is")
        for i in list(Q.keys()):
            print(i, "->", Q[i])
```

The display() function shows the elements in the queue. If isEmpty() is true, it prints “Queue is empty”. If isFull() is true, it prints all the elements of the queue; otherwise, it prints the elements in the queue.

Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert into Queue ")
    print("2. Delete from Queue ")
    print("3. Display the Queue")
```

```

print("4. Exit ")
opt = int(input("Enter a valid menu item ... "))
return opt

```

Program continues...

```

if __name__ == "__main__":
    size = int(input("Enter the size of the Queue "))
    Q, front, rear = create()
    i = 1
    while (i > 0 and i < 4 ):
        i = menu()
        if i == 1:
            Q, rear = enqueue(Q, front, rear, size)
            display(Q, rear, size)
        elif i == 2:
            Q, front = dequeue(Q, front, rear)
            if (front > rear):
                front = 0
                rear = -1
            display(Q, rear, size)
        elif i == 3:
            display(Q, rear, size)
        else:
            print("Exit")

```

The output of the program is the same as the previous program.

6.4.3 Implementation of queue using a class

Let us implement queue using a class.

Python program 4

```

# Creating an Queue
class Queue:
    def __init__(self, size):
        self.maxSize = size
        self.front = 0
        self.rear = -1
        self.Q = list()
        print("\nEmpty Queue Created")

```

The initializer takes the object and the size as the parameter to create an empty queue. It assigns front = 0 and rear = -1. It creates:

Program continues...

```
def isFull(self):
    #if (self.rear-self.front >= self.maxSize-1):
    if ( self. rear >= self.maxSize-1):
        return 1
    else:
        return 0
```

The method `isFull()` checks if the queue is full. It returns 1 if the queue is full and 0 otherwise.

Program continues...

```
def isEmpty(self):
    if (self.front == self.rear or self.rear == -1):
        return 1
    else:
        return 0
```

The method `isEmpty()` checks if the queue is empty. It returns 1 if the queue is empty and 0 otherwise.

Program continues...

```
def enqueue(self):
    if (self.isFull() == 1):
        print("Overflow: No more place for the Queue")
    else:
        no = int(input(("Enter a number to insert into the
                        Queue ")))
        self.rear += 1
        self.Q.append(no)
```

The `enqueue()` method enters an element into the queue. If the queue is not full, increment the rear by one and append the new element to the queue.

Program continues...

```
def dequeue(self):
    if (self.isEmpty()==1):
        print("Underflow: There is no element in the Queue")
    else:
        print("The element to Delete is ",self.Q[self.front])
        self.front += 1
```

The `dequeue()` method deletes an element from the queue. If the queue is not empty, increment the front by one.

Program continues...

```

def display(self):
    print("Front is at index -> ",self.front)
    print("Rear is at index -> ",self.rear)
    if(self.isEmpty()==1):
        print("\nQueue is Empty")

    elif(self.isFull()==1):
        print("The elements in the Queue are")
        for i in range(self.front,self.rear+1):
            print(i, "->",self.Q[i])
        print("\nQueue is Full")

    else:
        print("The elements in the Queue is")
        for i in range(self.front,self.rear+1):
            print(i, "->",self.Q[i])

```

The display() method shows all the elements in the queue.

Program continues...

```

def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert into Queue ")
    print("2. Delete from Queue ")
    print("3. Display the Queue")
    print("4. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt

```

Program continues...

```

if __name__ == "__main__":
    size = int(input("Enter the size of the Queue "))
    Qu = Queue(size)
    i=1
    while (i > 0 and i < 4 ):
        i = menu()
        if i == 1:
            Qu.enqueue()
            Qu.display()
        elif i == 2:
            Qu.dequeue()
            if (Qu.front > Qu.rear):
                Qu.front = 0
                Qu.rear = -1
            print()

```

```
    Qu.display()
elif i == 3:
    Qu.display()
else:
    print("Exit")
```

The output of the program is the same as the earlier program.

6.4.4 Implementation of queue using queue module

The queue module in Python can be used to generate a queue object. This is part of the Python standard library.

Python program 5

```
from queue import Queue
def enqueue(q):
    if (not q.full()):
        data = int(input(("Enter a number to insert into the Queue ")))
        q.put(data)
    else:
        print("There is NO SPACE in the Queue")
```

The enqueue() function first checks if the queue is full. This is done by the built-in method q.full(). Once this queue is full, insertion cannot be performed until all queue items have been deleted. If the queue is not full, insertion can be done by the q.get() method.

Program continues...

```
def dequeue(q):
    if (not q.empty()):
        data=q.get()
        print("The Item Deleted is : ",data)
    else:
        print("There is NO DATA in the queue")
```

The dequeue() function first checks if the queue is empty. This is done by the built-in method q.empty(). Deletion cannot be performed until there is at least one element in the queue. If the queue is not empty, deletion can be done by the q.get() method.

Program continues...

```
def Display(q):
    if(q.full()):
        print("The Elements in the Queue are : [ ",end="")
        for i in q.queue:
            print(i," ",end="")
```

```

    print("]")
    print("The Queue is Full")

elif(q.empty()):
    print("The Queue is EMPTY")
else:
    print("The Elements in the Queue are : [ ",end="")
    for i in q.queue:
        print(i," ",end="")
    print("]")

```

The Display() function shows all the elements in the queue. All the elements can be fetched from q.queue.

Program continues...

```

def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert to Queue ")
    print("2. Delete from Queue ")
    print("3. Display the Queue")
    print("4. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt

```

Program continues...

```

if __name__ == "__main__":
    size = int(input("Enter the size of the Queue "))
    Q = Queue(size)
    i=1
    while (i > 0 and i < 4 ):
        i = menu()
        if (i == 1):
            enqueue(Q)
            Display(Q)

        elif (i == 2):
            dequeue(Q)
            Display(Q)

        elif (i == 3):
            Display(Q)
        else:
            print("Exit")

```

In queue, size is an integer that specifies the maximum number of items that can be added to the queue. Once this size is achieved q.full(), insertion cannot be performed until all queue items have been deleted. The queue size is unlimited if it is less than or equal to zero.

Output

```
Enter the size of the Queue 3

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
The Elements in the Queue are : [ 10 ]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 20
The Elements in the Queue are : [ 10 20 ]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 30
The Elements in the Queue are : [ 10 20 30 ]
The Queue is Full

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : 10
The Elements in the Queue are : [ 20 30 ]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : 20
The Elements in the Queue are : [ 30 ]
```

```

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : 30
The Queue is EMPTY

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit

```

We can observe that except for the implementation of queue using the queue module, all the previous examples have one thing in common. Even though the queue had empty spaces, still we are not allowed to insert data into the queue as the rear of the queue is pointing to the last element (Figure 6.8a).

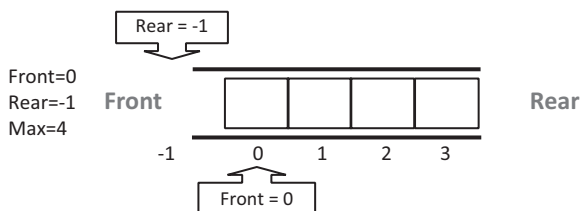


Figure 6.8a Empty queue

Insert an element 10 to the queue (Figure 6.8b).

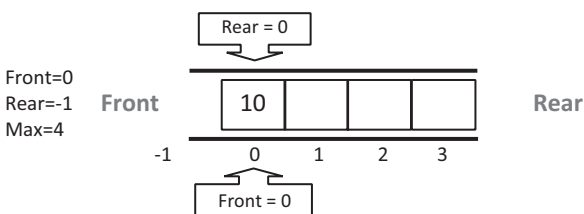


Figure 6.8b Insert of element 10 into queue

Insert an element 20 to the queue (Figure 6.8c).

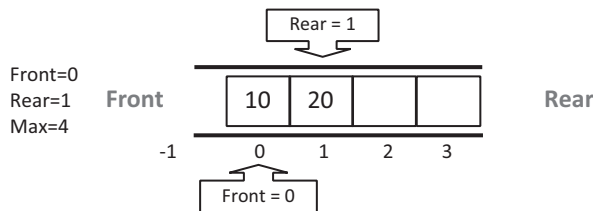


Figure 6.8c Insert of element 20 into queue

Insert an element 30 to the queue (Figure 6.8d).

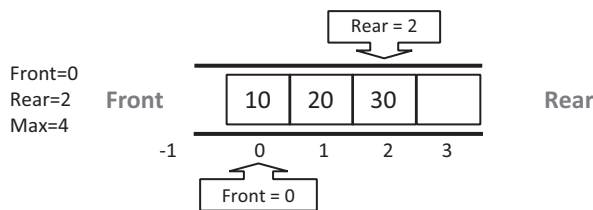


Figure 6.8d Insert of element 30 into queue

Insert an element 40 to the queue (Figure 6.8e).

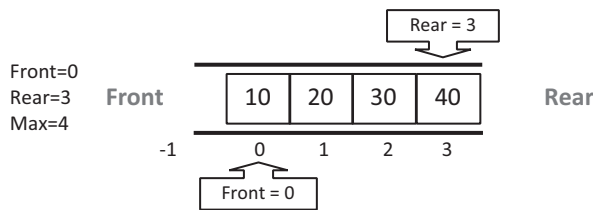


Figure 6.8e Insert of element 40 into queue

Now the queue is full. Delete an element from the queue (Figure 6.8f).

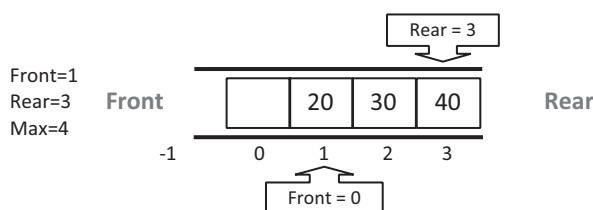


Figure 6.8f Deletion of element 10 from the queue

Now if we try to insert an element 50 to the queue, it will not be allowed since $\text{rear} = \text{max} - 1$ is true. which is the queue-full condition. Thus, even though there are many vacant cells, we may get a queue-full condition.

There are two ways to avoid this problem:

1. After each deletion, shift all the data elements to the left so that the actual front is always -1 , update rear to rear -1 so that $\text{rear} = \text{max} - 1$ is not always true. Obviously this is not a feasible solution since this involves a lot of frequent data movement and is time-consuming.
2. The other way is to choose alternate representation: either circular storage of data using the same array representation or use linked list representation.

6.5 CIRCULAR QUEUE

The data structure *circular queue* uses the same array representation but wraps around when a queue-full condition occurs using modular arithmetic. In other words, when the rear becomes equal to size, we reset rear to $\text{rear} = \text{rear} + 1 \% \text{size}$.

Using this representation, let us define the class MyCircularQueue.

Python program 6

```
class MyCircularQueue():
    def __init__(self, S):
        self.size = S
        self.queue = [None] * S
        self.front = self.rear = -1
```

The `__init__()` method initializes the circular queue. It creates an empty list of size S and initializes the front and rear to -1 .

Program continues...

```
# Insert an element into the circular queue
def enqueue(self):
    data = int(input(("Enter a number to insert into the
    Queue ")))
    if ((self.rear + 1) % self.size == self.front):
        print("The circular queue is full\n")

    elif (self.front == -1):
        self.front = 0
        self.rear = 0
        self.queue[self.rear] = data
    else:
        self.rear = (self.rear + 1) % self.size
        self.queue[self.rear] = data
```

The enqueue() method adds an element at the rear. When the first element is added, rear and front are assigned to 0. Every time an element is added to the circular queue, it calculates the rear by $(\text{self.rear} + 1) \% \text{self.size}$ and adds the element at the rear.

Let us assume the circular queue (Figure 6.9a).

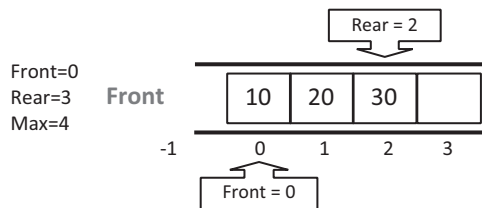


Figure 6.9a Initial circular queue

Let us add 40 to the circular queue. Calculate the index by $(\text{rear} + 1) \% \text{size}$, i.e., $(2+1) \% 4 = 3$. So insert 40 at index 3 (Figure 6.9b).

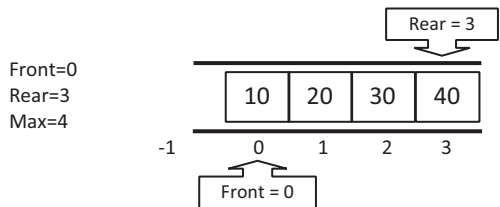


Figure 6.9b Insertion of 10 into a circular queue

Now delete an element from the circular queue (Figure 6.9c).

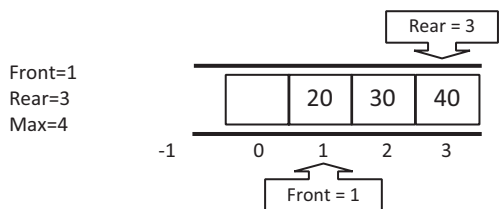


Figure 6.9c Deletion of element from circular queue

Now add one element 50 to the circular queue. Find the index by calculating $(\text{rear} + 1) \% \text{size}$, i.e., $(3+1) \% 4 = 0$. Add the new element 50 at the 0th index (Figure 6.9d).

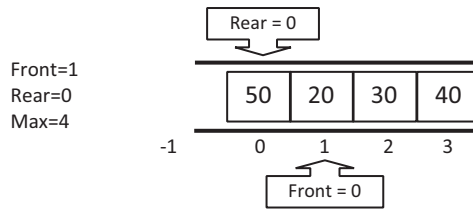


Figure 6.9d Insertion of element into circular queue

Program continues...

```
# Delete an element from the circular queue
def dequeue(self):
    if (self.front == -1):
        print("The circular queue is empty\n")

    elif (self.front == self.rear):
        self.queue[self.front] = None
        self.front = -1
        self.rear = -1
    else:
        self.queue[self.front] = None
        self.front = (self.front + 1) % self.size
```

The `dequeue()` method deletes an element from the circular queue that is at the front. When the front and rear are pointing to the same location, after deletion the circular queue becomes empty. So initialize the front and rear to `-1`. If not, the front is assigned to `None` and front is moved to `self.front = (self.front + 1) % self.size`.

Let us assume the circular queue (Figure 6.10a).

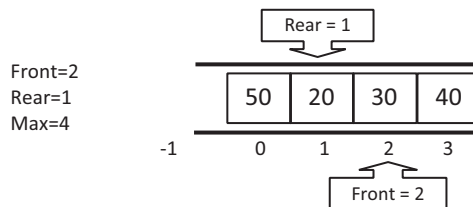


Figure 6.10a Initial circular queue

Now delete the element at the front of the circular queue. After deletion, the front points to the index $(\text{front}+1) \% \text{size}$, i.e., $(2+1) \% 4 = 3$ (Figure 6.10b).

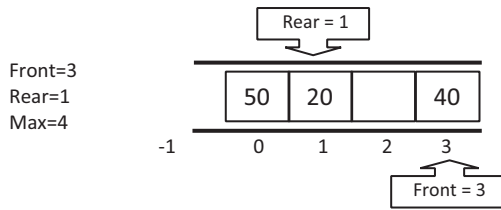


Figure 6.10b Deletion of element 30 from circular queue

Now delete the element at the front of the circular queue. After deletion, the front points to the index $(\text{front}+1) \% \text{size}$, i.e., $(3+1) \% 4 = 0$ (Figure 6.10c).

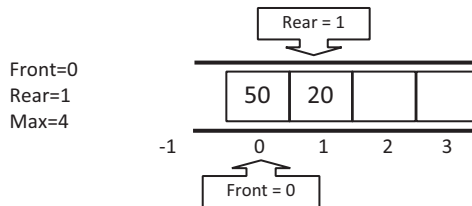


Figure 6.10c Deletion of element 40 from circular queue

Program continues...

```
def Display(self):
    print("Front is at index -> ",self.front)
    print("Rear is at index -> ",self.rear)
    if(self.front == -1):
        print("No element in the circular queue")
        print()

    elif((self.rear + 1) % self.size == self.front):
        print("Queue is Full")
        for i in range(0, self.size):
            print(self.queue[i], end=" ")
        print()

    else:
        for i in range(0, self.size):
            print(self.queue[i], end=" ")
        print()
```

The Display() method shows the elements in the circular queue. When front points to -1, it prints “No element in the circular queue”, and when $(\text{rear} + 1) \% \text{size} = \text{front}$, it shows the elements in the circular queue and prints

“Queue is Full”. In all other cases, it shows all the elements in the circular queue.

Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert to Circular Queue ")
    print("2. Delete from Circular Queue ")
    print("3. Display the Circular Queue")
    print("4. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

Program continues...

```
if __name__ == "__main__":
    Size = int(input("Enter the size of the Circular Queue "))
    CQ = MyCircularQueue(Size)
    i=1
    while (i > 0 and i < 4 ):
        i = menu()
        if (i == 1):
            CQ.enqueue()
            CQ.Display()

        elif (i == 2):
            CQ.dequeue()
            CQ.Display()

        elif (i == 3):
            CQ.Display()
        else:
            print("Exit")
```

Output

```
Enter the size of the Circular Queue 3
~~~ MENU ~~~
1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
Front is at index -> 0
Rear is at index -> 0
10 None None
```

~~~ MENU ~~~

1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 20

Front is at index -> 0

Rear is at index -> 1

10 20 None

~~~ MENU ~~~

1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 30

Front is at index -> 0

Rear is at index -> 2

Queue is Full

10 20 30

~~~ MENU ~~~

1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit

Enter a valid menu item ... 2

Front is at index -> 1

Rear is at index -> 2

None 20 30

~~~ MENU ~~~

1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit

Enter a valid menu item ... 2

Front is at index -> 2

Rear is at index -> 2

None None 30

~~~ MENU ~~~

1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 40

```
Front is at index -> 2
Rear is at index -> 0
40 None 30

~~~ MENU ~~~
1. Insert to Circular Queue
2. Delete from Circular Queue
3. Display the Circular Queue
4. Exit
Enter a valid menu item ... 4
Exit
```

6.6 DOUBLE-ENDED QUEUE

A double-ended queue is also known as a deque. It defines a data structure where elements can be inserted and deleted at both ends. It supports both stack-like and queue-like capabilities. This hybrid data structure provides all the features of stacks and queues in a single data structure (Figure 6.11).

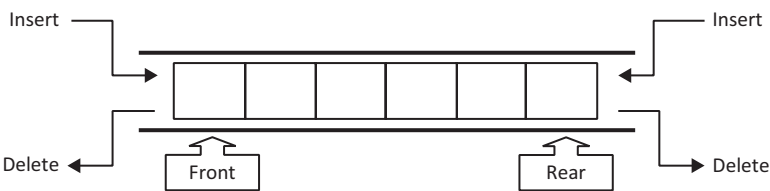


Figure 6.11 Double-ended queue

6.6.1 ADT of dequeue

Operations: See Table 6.4.

Table 6.4 Dequeue ADT operations

| Operation     | Meaning                                                  |
|---------------|----------------------------------------------------------|
| Create()      | Creates a deque that is empty                            |
| addFront(el)  | Adds a new element “el” to the front and returns nothing |
| addRear(el)   | Adds a new element “el” to the rear and returns nothing  |
| deleteFront() | Removes an element from the front and returns the item   |
| deleteRear()  | Removes an element from the rear and returns the item    |
| isEmpty()     | Checks if the deque is empty; returns 1 if true, else 0  |
| size()        | Returns the number of elements in the deque              |

**Axioms:** See Table 6.5.

Table 6.5 Axioms of dequeue operations

| Axiom                        | Returned value |
|------------------------------|----------------|
| isEmpty(create())            | True           |
| isEmpty(insertRear(el))      | False          |
| deleteFront(insertFront(el)) | El             |
| deleteFront(create())        | Error          |
| deleteRear(create())         | Error          |

As an example, let *d* be an empty deque (Table 6.6).

Table 6.6 Axioms of dequeue operations

| Deque operation | Deque content | Return value |
|-----------------|---------------|--------------|
| d.isEmpty()     | [ ]           | True         |
| d.addrear(4)    | [ 4 ]         | -            |
| d.addrear(5)    | [ 5 4 ]       |              |
| d.addFront(3)   | [ 5 4 3 ]     |              |
| d.deleterear()  | [ 4 3 ]       | 5            |
| d.deleteFront() | [ 4 ]         | 3            |
| d.isEmpty()     | [ 4 ]         | False        |

Deque can best be implemented using a doubly linked list.

The general deque class has some possible subtypes:

*Input-restricted deque* where deletions can be made from both ends and insertions can be made only from one end (Figure 6.12).

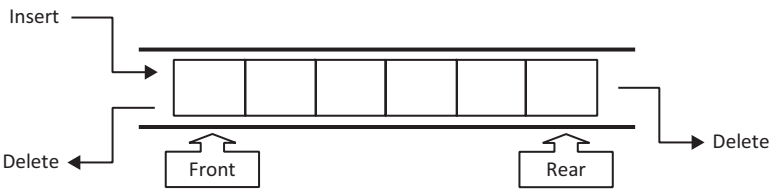


Figure 6.12 Input-restricted deque

*Output-restricted deque* where insertions can be made from both ends and deletions can be made from only one end (Figure 6.13).

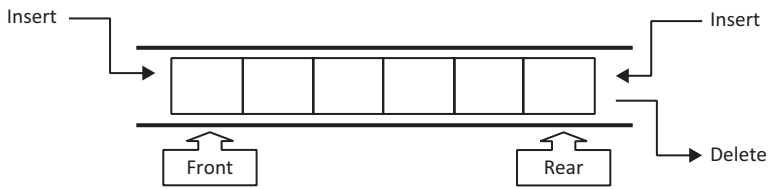


Figure 6.13 Output-restricted deque

The following Python program implements deque.

### Python program 7

```
import collections

def Insert(q):
 data = int(input(("Enter a number to insert into the
 Queue ")))
 side= input("Enter the side (F-> at front and R -> at
 rear) : ")
 if (side == 'F' or side == 'f'):
 q.appendleft(data)
 else:
 q.append(data)
```

In Python, collections are simply container data types, such as lists, sets, tuples and dictionaries.

Python's collections module implements specialized data structures as an alternative to the language's built-in container data types. Appends and pops from either end of a list are supported by a deque object. It uses less memory than a regular list object. When an item is removed from a regular list object, all items to the right are relocated one index to the left. As a result, it is extremely slow.

The Insert() function adds elements at both ends. There are two specialized methods, appendleft() and append(), available with the deque(). The function appendleft() adds an element at the front, whereas append() adds the element at the rear.

### Program continues...

```
def Delete(q):
 if (len(q) != 0):
 side= input("Enter from where to Delete (F-> at front and
 R -> at rear) : ")
 if (side == 'F' or side == 'f'):
 item = q.popleft()
 print("The element deleted is : ", item)
```

```
 else:
 item =q.pop()
 print("The element deleted is : ", item)
 else:
 print("There is NO DATA in the queue")
```

The Delete() function deletes elements from both ends. There are two specialized methods popleft() and pop(), available with the deque(). The function popleft() deletes the element at the front, whereas pop() deletes the element from the rear.

### Program continues...

```
def Display(q):
 if(len(q) == 0):
 print("The Queue is EMPTY")
 else:
 print("The Elements in the Queue are : [",end="")
 for i in q:
 print(i," ",end="")
 print("]")
```

### Program continues...

```
def menu():
 print("\n~~~ MENU ~~~ ")
 print("1. Insert to Queue ")
 print("2. Delete from Queue ")
 print("3. Display the Queue")
 print("4. Exit ")
 opt = int(input("Enter a valid menu item ... "))
 return opt
```

### Program continues...

```
if __name__ == "__main__":
 Q = collections.deque()
 i=1
 while (i > 0 and i < 4):
 i = menu()
 if (i == 1):
 Insert(Q)
 Display(Q)
 elif (i == 2):
 Delete(Q)
 Display(Q)
```

```
elif (i == 3):
 Display(Q)
else:
 print("Exit")
```

## Output

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 10

Enter the side (F-> at front and R -> at rear) : f

The Elements in the Queue are : [ 10 ]

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 20

Enter the side (F-> at front and R -> at rear) : f

The Elements in the Queue are : [ 20 10 ]

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Queue 30

Enter the side (F-> at front and R -> at rear) : r

The Elements in the Queue are : [ 20 10 30 ]

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit

Enter a valid menu item ... 2

Enter from where to Delete (F-> at front and R -> at rear) : f

The element deleted is : 20

The Elements in the Queue are : [ 10 30 ]

~~~ MENU ~~~

1. Insert to Queue
2. Delete from Queue

```
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
Enter from where to Delete (F-> at front and R -> at rear) : r
The element deleted is : 30
The Elements in the Queue are : [10]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
Enter from where to Delete (F-> at front and R -> at rear) : R
The element deleted is : 10
The Queue is EMPTY

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit
```

## 6.7 PRIORITY QUEUE

The use of a queue data structure ensures that items are processed in the order in which they are received.

Consider the setting of a hospital. In most cases, patients are treated in the order that they arrive. There may be times, however, when the first in, first out rule cannot be rigidly followed. For example, FIFO cannot be used if a patient arrives in a life-threatening condition following an accident. He must be attended immediately on priority basis.

In the world of computer science, an interactive program's print request takes precedence over a batch-processing program's print request. There are numerous instances when a work or a process is given a priority. Customers and processes with higher importance are pushed to the front of the queue using a particular data structure called a *priority queue*. A priority queue can be implemented in two ways. Let's give each element a priority (integer value); the smaller the integer, the greater the priority.

1. Maintain a separate queue of all the elements with the same priority. Let us assume there are jobs with three different priorities. Then there will be three different queues as shown in Figure 6.14.

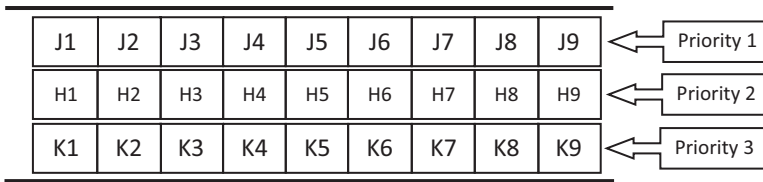


Figure 6.14 Priority queue

The jobs with priority 2 will be processed only when all the jobs of priority 1 are processed and the queue is empty. Jobs with priority 3 will be processed when the queues of priority 1 and 2 are empty. If higher priority jobs keep on coming, the lower priority jobs get delayed indefinitely.

2. The second method is to store all jobs as structures together with their priorities. All the jobs are stored as a queue of elements, with the highest priority job always in the front. When a new job arrives, it is inserted at the appropriate position in the queue, based on its priority.

When we store the jobs (elements) in a sorted list, we can easily delete them because the job with the highest priority is always at the front. However, putting the job in the right place is difficult since we must first locate it in the list.

If we keep the elements as an unsorted list, insertion is easy, since it can be put at the end of the list. However, deletion is difficult, since from the unordered list we have to pick up the job with highest priority.

### 6.7.1 Implementation of priority queue using class

We will make use of Section 6.4.3 where we implemented a queue using a class.

#### Python program 8

```
Creating an Empty Priority Queue
class Queue:
 def __init__(self,size):
 self.maxSize = size
 self.front=0
 self.rear = -1
 self.priority= 0
 self.PQ = list()
 print("\nEmpty Queue Created")
```

The `__init__()` method initializes the priority queue as discussed in Section 6.4.3, apart from that a new attribute `priority` is assigned to 0. The priority queue is a list of tuples.

**Program continues...**

```
def isFull(self):
 if (self.rear >= self.maxSize-1):
 return 1
 else:
 return 0
```

**Program continues...**

```
def isEmpty(self):
 if (self.front > self.rear or self.rear == -1):
 return 1
 else:
 return 0
```

The methods `isFull()` and `isEmpty()` are as described earlier.

**Program continues...**

```
def enqueue(self):
 if (self.isFull() == 1):
 print("Overflow: No more place for the Queue")
 else:
 no = int(input(("Enter a number to insert into the\nQueue ")))
 p = int(input(("Enter priority of data (0 -> Low, 1->\nHigh) ")))
 while (p not in [0,1]):
 p = int(input(("Enter priority of data (0 -> Low, 1->\nHigh) ")))
 if (len(self.PQ) == 0):
 temp=(p,no)
 self.PQ.append(temp)
 elif (p ==1):
 j=0
 while (self.PQ[j][0] != 0 and len(self.PQ) > j):
 j+=1
 temp=(p,no)
 self.PQ.insert(j,temp)
 else:
 temp=(p,no)
 self.PQ.append(temp)
 self.rear +=1
```

The enqueue() method adds an element to the priority queue. Along with the data to be inserted, we need to insert the priority with value 0 for low priority and 1 for high priority. This forms a tuple. Inserting the tuple into the priority queue has three cases.

Case 1: When there is no data in the priority queue, the tuple of the key and value pair is appended to the priority queue.

Case 2: When the priority queue contains some data, the tuple with priority 0 is appended to the queue.

Case 3: When a tuple with priority 1 is inserted, it is inserted before all the tuples having priority 0 but after any other tuple with priority 1.

**Program continues...**

```
def dequeue(self):
 if (self.isEmpty()==1):
 print("Underflow: There is no element in the Queue")
 else:
 print("The element to Delete is ",self.PQ[self.front])
 self.front += 1
```

The dequeue() method deletes an element that is at the front of the queue. It simply increments the front by one.

**Program continues...**

```
def display(self):
 print("Front is at index -> ",self.front)
 print("Rear is at index -> ",self.rear)
 if(self.isEmpty()==1):
 print("\nQueue is Empty")
 elif(self.isFull()==1):
 print("The elements in the Queue are")
 for i in range(self.front,self.rear+1):
 print(i, "->",self.PQ[i])
 print("\nQueue is Full")
 else:
 print("The elements in the Queue is")
 for i in range(self.front,self.rear+1):
 print(i, "->",self.PQ[i])
```

**Program continues...**

```
def menu():
 print("\n~~~ MENU ~~~ ")
 print("1. Insert into Queue ")
 print("2. Delete from Queue ")
 print("3. Display the Queue")
 print("4. Exit ")
 opt = int(input("Enter a valid menu item ... "))
 return opt
```

**Program continues...**

```
if __name__ == "__main__":
 size = int(input("Enter the size of the Queue "))
 Qu = Queue(size)
 i=1
 while (i > 0 and i < 4):
 i = menu()
 if i == 1:
 Qu.enqueue()
 Qu.display()
 elif i == 2:
 Qu.dequeue()
 if (Qu.front > Qu.rear):
 Qu.front = 0
 Qu.rear = -1
 print()
 Qu.display()
 elif i == 3:
 Qu.display()
 else:
 print("Exit")
```

**Output**

```
Enter the size of the Queue 3
Empty Queue Created
~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
Enter priority of data (0 -> Low, 1-> High ) 0
Front is at index -> 0
```

---

```
Rear is at index -> 0
The elements in the Queue is
0 -> (0, 10)

~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 20
Enter priority of data (0 -> Low, 1-> High) 1
Front is at index -> 0
Rear is at index -> 1
The elements in the Queue is
0 -> (1, 20)
1 -> (0, 10)

~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 30
Enter priority of data (0 -> Low, 1-> High ) 0
Front is at index -> 0
Rear is at index -> 2
The elements in the Queue are
0 -> (1, 20)
1 -> (0, 10)
2 -> (0, 30)

Queue is Full

~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to Delete is (1, 20)
Front is at index -> 1
Rear is at index -> 2
The elements in the Queue are
1 -> (0, 10)
2 -> (0, 30)

Queue is Full
```

```
~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to Delete is (0, 10)
Front is at index -> 2
Rear is at index -> 2
The elements in the Queue are
2 -> (0, 30)

Queue is Full

~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The element to Delete is (0, 30)

Front is at index -> 0
Rear is at index -> -1

Queue is Empty

~~~ MENU ~~~
1. Insert into Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit
```

### 6.7.2 Implementation of priority queue by importing PriorityQueue package of queue module

Here we have used the PriorityQueue package available in the queue module.

#### Python program 9

```
from queue import PriorityQueue

def enqueue(q):
    data = input(("Enter a number to insert into the Queue "))
    p = int(input(("Enter priority of data (0 -> Highest
        Priority ) ")))
    temp = (p,data)
    q.put(temp)
```

The queue module inserts a tuple (priority, data) by the *put()* method. The tuple having priority 0 has the highest priority.

### Program continues...

```
def dequeue(q):
    if (not q.empty()):
        next_item = q.get()
        print("The Item Deleted is : ", next_item)
    else:
        print("Priority queue is Empty")
```

The queue module deletes the tuple (priority, data) with high priority data that is at the front of the queue by the *get()* method.

### Program continues...

```
def Display(q):
    print(q.queue)
```

### Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Insert to Queue ")
    print("2. Delete from Queue ")
    print("3. Display the Queue")
    print("4. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

### Program continues...

```
if __name__ == "__main__":
    PQ = PriorityQueue()
    i=1
    while (i > 0 and i < 4 ):
        i = menu()
        if (i == 1):
            enqueue(PQ)
            Display(PQ)
        elif (i == 2):
            dequeue(PQ)
            Display(PQ)
        elif (i == 3):
            Display(PQ)
        else:
            print("Exit")
```

PriorityQueue() creates a priority queue that is imported from the queue module. An integer can be passed to specify the maximum number of items that can be added to the queue. Once this size is attained, insertion will be halted until all queue items have been consumed. The queue size is unlimited if the integer passed is less than or equal to zero.

## Output

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
Enter priority of data (0 -> Highest Priority) 1
[(1, '10')]
```

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 20
Enter priority of data (0 -> Highest Priority ) 0
[(0, '20'), (1, '10')]
```

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 30
Enter priority of data (0 -> Highest Priority) 1
[(0, '20'), (1, '10'), (1, '30')]
```

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (0, '20')
[(1, '10'), (1, '30')]
```

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
```

```

4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (1, '10')
[(1, '30')]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (1, '30')
[]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit

```

### 6.7.3 Implementation of priority queue using heapq module

A priority queue is also implemented in Python by the heapq module. The tuple having priority 0 has the highest priority.

#### Python program 10

```

import heapq

def menu():
 print("\n~~~ MENU ~~~ ")
 print("1. Insert to Queue ")
 print("2. Delete from Queue ")
 print("3. Display the Queue")
 print("4. Exit ")
 opt = int(input("Enter a valid menu item ... "))
 return opt

```

#### Program continues...

```

if __name__ == "__main__":
 PQ = []
 i=1
 while (i > 0 and i < 4):
 i = menu()
 if (i == 1):
 data = input(("Enter a number to insert into the Queue "))

```

```
p = int(input(("Enter priority of data (0 -> Highest
Priority) ")))
temp = (p,data)
heapq.heappush(PQ, (temp))
heapq.heapify(PQ)
print(PQ)
elif (i == 2):
 data=heapq.heappop(PQ)
 print("The Item Deleted is : ",data)
 if(PQ):
 print(PQ)
 else:
 print("No Data in Priority Queue")
elif (i == 3):
 print(PQ)
else:
 print("Exit")
```

The `heapq` module has a `heappush()` method to add a tuple with priority and data. Every time we insert a tuple, it is appended to the heap. So, after every insertion we need to apply the `heapify()` method to arrange the tuples in descending order of priority. The `heappop()` method deletes tuples from the front of the priority queue.

## Output

```
~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 10
Enter priority of data (0 -> Highest Priority ) 2
[(2, '10')]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 20
Enter priority of data (0 -> Highest Priority) 1
[(1, '20'), (2, '10')]

~~~ MENU ~~~
1. Insert to Queue
2. Delete from Queue
3. Display the Queue
```

```

4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Queue 30
Enter priority of data (0 -> Highest Priority ) 0
[(0, '30'), (2, '10'), (1, '20')]

```

```

~~~ MENU ~~~

```

```

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (0, '30')
[(1, '20'), (2, '10')]

```

```

~~~ MENU ~~~

```

```

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (1, '20')
[(2, '10')]

```

```

~~~ MENU ~~~

```

```

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 2
The Item Deleted is : (2, '10')
No Data in Priority Queue

```

```

~~~ MENU ~~~

```

```

1. Insert to Queue
2. Delete from Queue
3. Display the Queue
4. Exit
Enter a valid menu item ... 4
Exit

```

### True-false questions

1. Queues are also known as LIFO data structures.
  - a. True
  - b. False
2. A printer uses stack data structures.
  - a. True
  - b. False
3. While inserting an element onto a queue, we need to increment the front by one.
  - a. True
  - b. False

4. While deleting an element onto a queue, we need to increment the front by one.
  - a. True
  - b. False
5. A queue is empty when the front is greater than the rear.
  - a. True
  - b. False

### Fill-in-the-blank questions

1. The \_\_\_\_\_ queue adds and deletes elements at both ends.
2. To add an element to a queue we have the \_\_\_\_\_ operation.
3. Before inserting an element to a queue, the \_\_\_\_\_ condition is checked.
4. The isEmpty condition is checked before the \_\_\_\_\_ operation.
5.  $(Rear + 1) \% size == front$  is used to check the queue full condition in the \_\_\_\_\_ queue.

### Multiple-choice questions

1. A linear list of elements in which deletion can be done from one end (front) and insertion can take place only at the other end is known as a
  - a. Queue
  - b. Stack
  - c. Tree
  - d. Branch
2. The end at which a new element gets added to a queue is called the
  - a. Front
  - b. Rear
  - c. Top
  - d. Bottom
3. With what data structure can a priority queue be easily implemented?
  - a. Array
  - b. List
  - c. Heap
  - d. All of the above
4. What are the advantages of priority queues?
  - a. Easy to implement
  - b. Processes with different priority can be efficiently handled
  - c. Applications with differing requirements
  - d. All of the above

5. Let the following circular queue accommodate a maximum six elements with the following data:  
front = 2 rear = 4  
queue = \_\_\_\_\_; L, M, N, \_\_\_\_\_, \_\_\_\_\_  
What will happen after the ADD O operation takes place?
- a. front = 2 rear = 5  
queue = \_\_\_\_\_; L, M, N, O, \_\_\_\_
- b. front = 3 rear = 5  
queue = L, M, N, O, \_\_\_\_
- c. front = 3 rear = 4  
queue = \_\_\_\_\_; L, M, N, O, \_\_\_\_
- d. front = 2 rear = 4  
queue = L, M, N, O, \_\_\_\_
6. If the elements A, B, C and D are placed in a queue and are deleted one at a time, in what order will they be removed?
- a. ABCD
- b. DCBA
- c. DCAB
- d. ADBC
7. When several jobs arrive at the printer, the jobs join the
- a. Queue
- b. Dequeue
- c. Stack
- d. None of the above
8. A data structure in which elements can be inserted or deleted at/from both ends but not in the middle is a \_\_\_\_\_.
- a. Queue
- b. Circular queue
- c. Dequeue
- d. Priority queue
9. A circular queue is implemented using an array of size 10. The array index starts with 0, front is 6, and rear is 9. The insertion of the next element takes place at the array index.
- a. 0
- b. 7
- c. 9
- d. 10
10. A normal queue of size MAX gets full when
- a.  $\text{Rear} = \text{MAX} - 1$
- b.  $\text{Rear} = \text{front}$
- c.  $\text{Rear} = \text{front} - 1$
- d.  $\text{Rear} = (\text{rear} + 1) \% \text{MAX}$
11. A queue can be implemented using
- a. Arrays
- b. Linked lists
- c. Two stacks
- d. All the above

12. A circular queue of size MAX gets full when
- a.  $\text{Rear} = \text{MAX} - 1$
  - b.  $\text{Front} = \text{Rear} - 1$
  - c.  $\text{Rear} = \text{Front} - 1$
  - d.  $\text{Front} = (\text{Rear} + 1) \% \text{MAX}$
13. While implementing propriety queue using the heapq module, thr highest priority is
- a. -1
  - b. 1
  - c. 0
  - d. 2
14. The minimum number of stacks needed to implement a queue is
- a. 1
  - b. 2
  - c. 3
  - d. 4
15. Which of the following is true with respect to a double-ended queue?
- a. Deletion is done only at the rear.
  - b. Insertion is only done at the rear.
  - c. Deletion is done only at the front.
  - d. Insertion and deletion can be done at both ends.
16. Which of the following not an application of priority queue?
- a. Interrupt handling in OS
  - b. Undo operation in a text editor
  - c. Huffman code
  - d. Editing in a text editor
17. Which if the following is an advantage of a double-ended queue?
- a. Can be used for both stack and queue
  - b. To find the longest sub-string in a text
  - c. To avoid collision in a hash table
  - d. Job scheduling algorithm
18. Breadth-First Search of a graph uses a
- a. Stack
  - b. Queue
  - c. Array
  - d. None of the above
19. A list of size 10 is used to build a circular queue. The list index begins with 0 and ends with 9. The front index is 6, and the rear index is 9. The list index determines where the next element is inserted.
- a. 0
  - b. 7
  - c. 9
  - d. 10

20. Priority queue can be implemented using

- |                 |                     |
|-----------------|---------------------|
| a. Dictionary   | c. Heapq module     |
| b. Queue module | d. All of the above |

### Descriptive questions

1. What is a circular queue? Write a Python program to insert an element in the circular queue. Write a Python function for printing the elements of the queue in reverse order.
2. Suppose the size of an array for implementing a circular queue is 100. Suppose the front is 50 and the rear is 99.
  - a. What are the values of front and rear after adding an element?
  - b. What are the values of rear and front after removing an element?
3. What are the advantages of circular queue over linear queue?
4. Write a Python program to create a queue using stack.
5. Write a Python program to find the number of elements in a queue.
6. Write a Python program that permits insertion at any vacant location at rear end.
7. Explain priority queue with the help of an example.
8. Explain how a circular queue is better than a linear queue.

### Answers to true–false questions

1. False
2. True
3. False
4. True
5. True

### Answers to fill-in-the-blank questions

1. double-ended
2. enqueue
3. isFull
4. deletion
5. circular

### Answers to multiple-choice questions

- |       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. a  | 2. b  | 3. d  | 4. d  | 5. a  |
| 6. a  | 7. a  | 8. c  | 9. a  | 10. a |
| 11. d | 12. d | 13. c | 14. b | 15. d |
| 16. b | 17. c | 18. b | 19. a | 20. d |